YMÜ227 NESNE TABANLI PROGRAMLAMA

PROJE RAPORU

Proje Adı: Kargo Otomasyonu(Java)

Ders Adı:	YMÜ227 Nesne Tabanlı Programlama
Öğretim Elemanı:	Sinem AKYOL / Nigar ÖZBEY
Öğrenci Adı Soyadı:	Abdullah Batur DEMİR
Öğrenci Numarası:	240290040
Teslim Tarihi:	04.01.2026

İçindekiler

1. Giriş
1.1. Problem Tanımı
1.2. Projenin Amacı
2. Önerilen Sistem Tasarımı
2.1. Gereksinimler ve Kısıtlar
2.2. Use-Case Diyagramı
2.3. Sequence Diyagramları
2.4. Sınıf Diyagramı
3. Sonuç

---

1. Giriş

Kargo ve lojistik sektörü, verilerin hızlı ve hatasız işlenmesini gerektiren dinamik bir alandır. Bu projede, kargo gönderim süreçlerinin dijital ortamda yönetilmesini sağlayan masaüstü tabanlı bir otomasyon sistemi geliştirilmiştir.

1.1. Problem Tanımı

Geleneksel veya manuel yöntemlerle yapılan kargo takiplerinde şu problemler ortaya çıkmaktadır:

- **Veri Karmaşası:** Gönderici, alıcı ve kurye bilgilerinin kağıt üzerinde veya düzensiz listelerde tutulması veri kaybına yol açmaktadır.

- **Takip Zorluğu:** Kargonun anlık durumunun (hazırlanıyor, dağıtımda, teslim edildi vb.) takibi zorlaşmaktadır.

- **Hatalı Girişler:** Kargo kayıtları sırasında insan hatasına bağlı eksik veya yanlış bilgi girişi yapılabilmektedir.

- **Zaman Kaybı:** Operatörlerin kurye ataması ve adres işlemlerini manuel yapması operasyonel süreci yavaşlatmaktadır.

1.2. Projenin Amacı

Bu projenin temel amacı, kargo kabul, atama ve takip süreçlerini dijitalleştirerek hızlandırmaktır. Proje sonunda elde edilecek kazanımlar şunlardır:

- Kargo verilerinin (Gönderici, Alıcı, Adres) standart bir yapıda kayıt altına alınması.

- Nesne Yönelimli Programlama (OOP) prensipleri kullanılarak sürdürülebilir bir kod yapısı oluşturulması.

- Kullanıcı dostu bir Grafik Arayüz (GUI) ile operatör hatalarının en aza indirilmesi.

- Kurye atamalarının ve kargo durum güncellemelerinin anlık olarak yapılabilmesi.

2. Önerilen Sistem Tasarımı

Sistem, Java programlama dili kullanılarak, Swing kütüphanesi ile görsel arayüz destekli ve OOP prensiplerine (Kalıtım, Kapsülleme) sadık kalınarak tasarlanmıştır.

2.1. Gereksinimler ve Kısıtlar

Fonksiyonel Gereksinimler:

- **FR-01:** Sistem, gönderici ve alıcı bilgilerini (Ad, Telefon) alarak yeni bir kargo kaydı oluşturabilmelidir.
- **FR-02:** Kullanıcı, sistemde tanımlı kuryelerden birini seçerek kargoya atayabilmelidir.
- **FR-03:** Sistem, kayıtlı tüm kargoları ID, Gönderici, Alıcı ve Durum bilgileriyle listeleyebilmelidir.
- **FR-04:** Kullanıcı, seçili bir kargonun durumunu (örn: "Yola Çıktı", "Teslim Edildi") güncelleyebilmelidir.
- **FR-05:** Kullanıcı, hatalı girilen veya iptal edilen kargo kayıtlarını sistemden silebilmelidir.
- **FR-06:** Sistem, sayısal alanlara (ID, Telefon) metin girilmesini engellemeli veya uyarı vermelidir.

Fonksiyonel Olmayan Gereksinimler:

- **Kullanılabilirlik:** Arayüz, sekmeli yapı (TabbedPane) kullanılarak "Kayıt" ve "Takip" işlemlerini birbirinden ayırmalıdır.
- **Performans:** Veriler bellekte (RAM) tutularak işlemler anlık gerçekleşmelidir.
- **Güvenilirlik:** Hatalı veri girişlerinde program çökmemeli, kullanıcıya `JOptionPane` ile uyarı mesajı verilmelidir.

Kısıtlar:

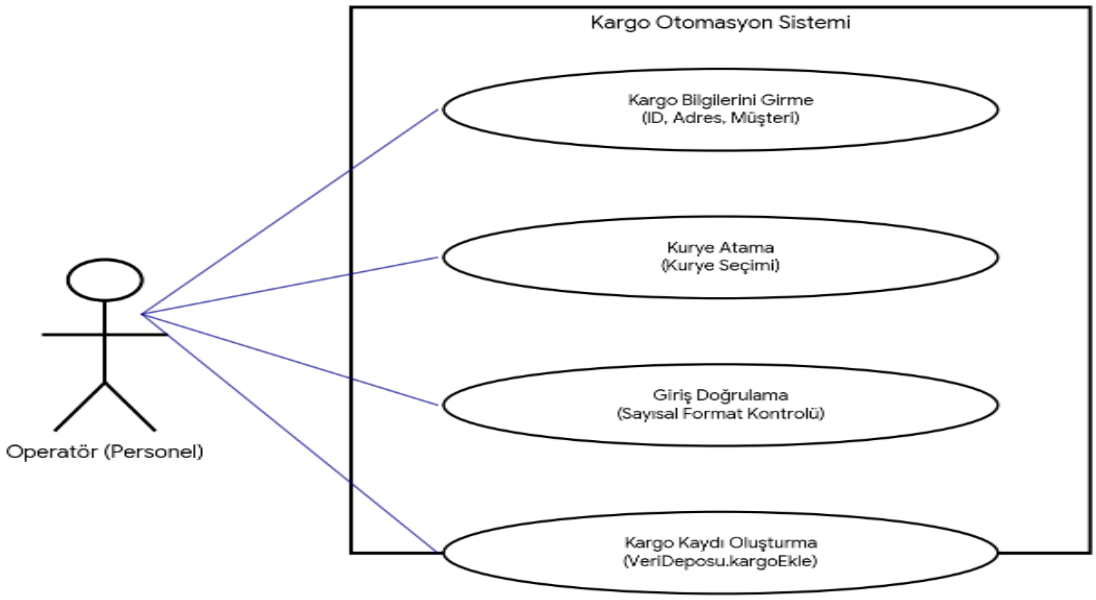
- Programlama Dili: Java
- Arayüz Teknolojisi: Java Swing (GUI)
- Veri Saklama: Program çalışma süresince `ArrayList` yapısında tutulacaktır (Veritabanı kullanılmayacaktır).

2.2. Use-Case Diyagramı

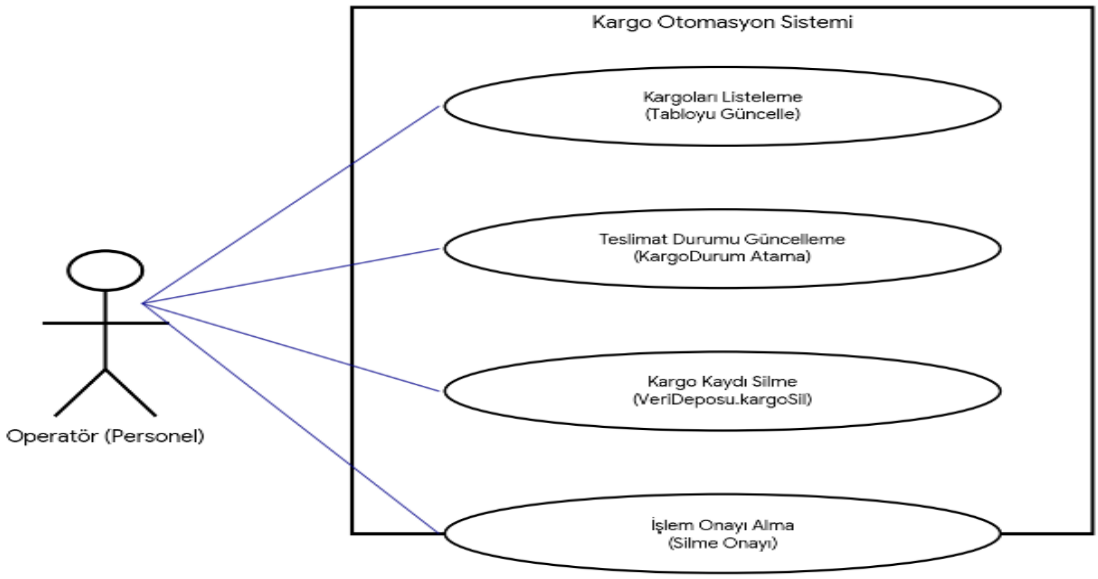
Sistemin ana aktörü **Operatör** (Kargo Görevlisi)'dür. Temel kullanım durumları şunlardır:

1. **Kargo Kaydı Oluştur:** Müşteri ve adres bilgilerinin girilmesi.
2. **Kurye Ata:** Listeden uygun kuryenin seçilmesi.
3. **Kargo Listele:** Mevcut kargoların görüntülenmesi.
4. **Durum Güncelle:** Kargo statüsünün değiştirilmesi.
5. **Kargo Sil:** Kaydın kaldırılması.

Use Case Diyagramı 1: Kargo Kayıt ve Operasyon



Use Case Diyagramı 2: Takip ve Yönetim

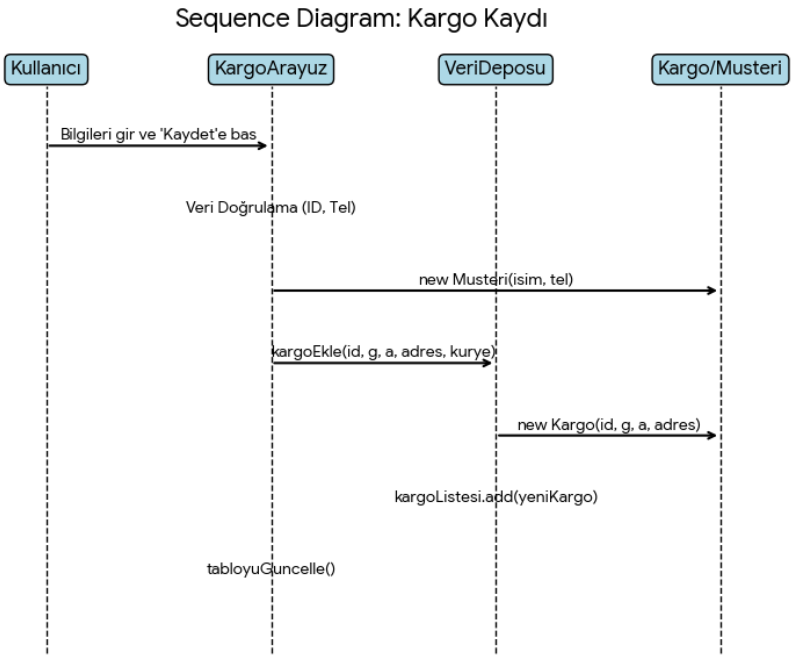


2.3. Sequence Diyagramları

Senaryo 1: Yeni Kargo Kaydı

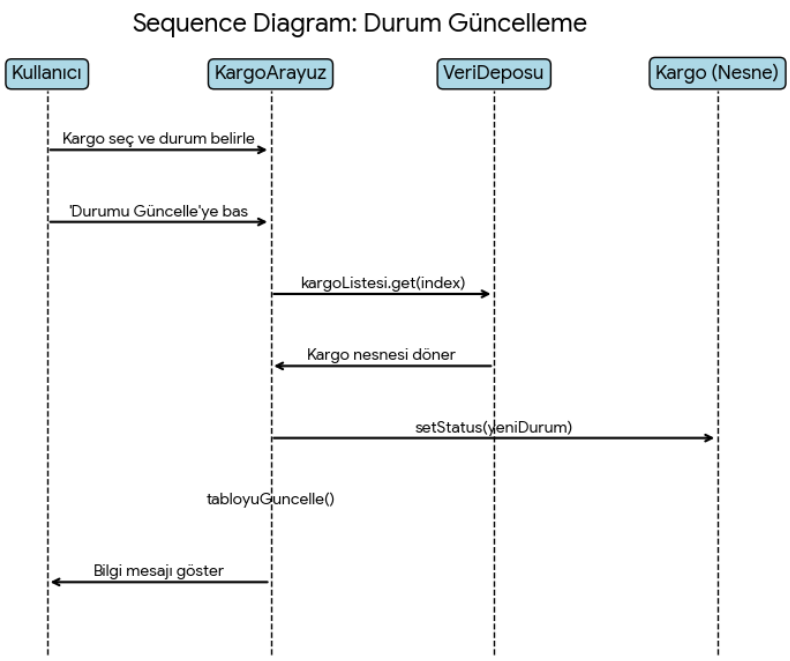
Kullanıcının arayüz üzerinden veri girmesi ve sistemin bu veriyi nesneye dönüştürüp kaydetmesi sürecidir.

1. Kullanıcı bilgileri girer ve "Kaydet" butonuna basar.
2. `KargoArayuz`, girişlerin boş olup olmadığını ve formatını kontrol eder.
3. Doğrulama başarılıysa `Musteri` nesneleri oluşturulur.
4. `VeriDeposu.kargoEkle()` metodu çağrılır.
5. Yeni `Kargo` nesnesi oluşturulur ve listeye eklenir.
6. Kullanıcıya "Başarılı" mesajı gösterilir.



Senaryo 2: Kargo Durum Güncelleme

1. Kullanıcı tablodan bir satır seçer.
2. Combo box üzerinden yeni durumu seçer ve "Güncelle" butonuna basar.
3. Sistem seçili satırın indeksini alır.
4. `VeriDeposu` üzerinden ilgili `Kargo` nesnesi bulunur.
5. Nesnenin `setStatus()` metodu çağrılır.
6. Tablo (`DefaultTableModel`) yenilenerek güncel durum ekrana yansıtılır.



2.4. Sınıf Diyagramı

Sistemde kullanılan sınıflar ve aralarındaki ilişkiler aşağıda tanımlanmıştır:

- **Kisi (Abstract Class):** `name` ve `phone` özelliklerini tutar. `Musteri` ve `Kurye` bu sınıftan türetilmiştir (Inheritance).

- **Musteri:** Kargo gönderen ve alan kişileri temsil eder. `Kisi` sınıfını genişletir.

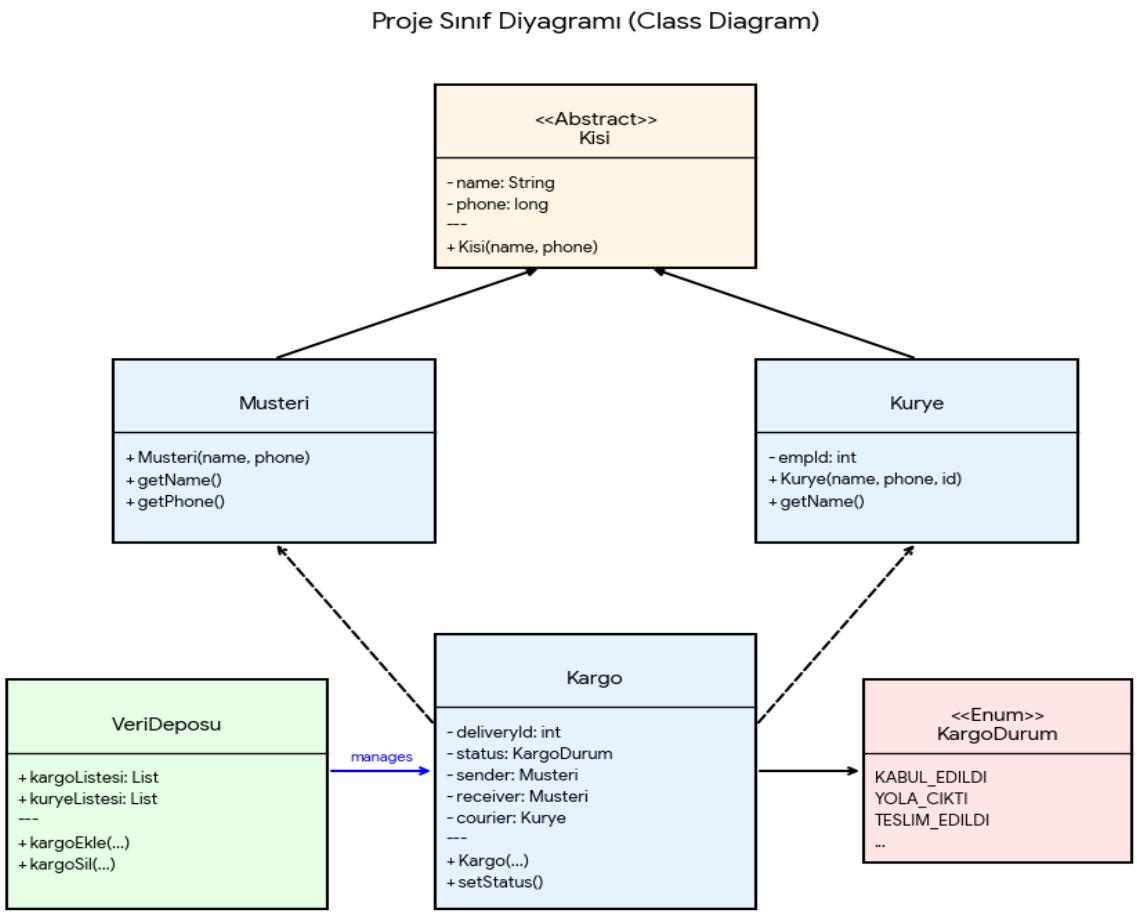
- **Kurye:** Kargo taşıyan personeli temsil eder. Ek olarak `empId` (Çalışan ID) tutar.

- **Kargo:** Sistemin ana varlığıdır. İçinde `Musteri` (gönderici/alıcı), `Kurye` ve `KargoDurum` nesnelerini barındırır (Association/Composition).

- **KargoDurum (Enum):** Kargo durumlarının sabitlerini (Teslim Edildi, Yola Çıktı vb.) tutar.

- **VeriDeposu:** Tüm kargo ve kurye listelerini `public static List` olarak tutan veri erişim sınıfıdır.

- **KargoArayuz:** Kullanıcı etkileşimini sağlayan `JFrame` sınıfıdır. `VeriDeposu`'na bağımlıdır.



3. Sonuç

Bu projede, bir kargo firmasının temel operasyonel ihtiyaçlarını karşılayan nesne tabanlı bir otomasyon sistemi geliştirilmiştir.

Tasarım Kararları ve Gerekçeler:

- **Kalıtım (Inheritance):** `Musteri` ve `Kurye` sınıflarında kod tekrarını önlemek için `Kisi` adında soyut bir üst sınıf oluşturulmuştur. Bu sayede isim ve telefon gibi ortak alanlar tek merkezden yönetilmiştir.
- **Enum Kullanımı:** Kargo durumları metin (String) olarak değil, `Enum` yapısı ile tutulmuştur. Bu sayede kod içerisinde "Yolda", "yola cıktı" gibi yazım hatalarının önüne geçilmiş ve tip güvenliği sağlanmıştır.
- **Statik Veri Deposu:** Veritabanı kullanılmadığı için veriler `VeriDeposu` sınıfında statik listelerde tutulmuştur. Bu yöntem, uygulamanın herhangi bir yerinden verilere kolayca erişilmesini sağlamıştır.
- **Swing GUI:** Konsol uygulaması yerine görsel arayüz tercih edilerek son kullanıcı deneyimi iyileştirilmiştir.

Geliştirilebilecek Yönler:

1. **Veritabanı Entegrasyonu:** Mevcut sistem verileri RAM'de tutmaktadır. MySQL veya SQLite entegrasyonu ile verilerin kalıcı olması sağlanabilir.
2. **Kullanıcı Giriş Paneli:** Yönetici ve Kurye girişleri ayrılarak yetkilendirme (Auth) sistemi eklenebilir.
3. **ID ile Arama Özelliği:** Kargoların listelendiği tabloda bulunmak istenen kargoyu ID’si ile arayarak kolayca görüntülenebilmesi sağlanabilir.
4. **Harita Entegrasyonu:** Kargonun teslimat adresinin harita üzerinde gösterilmesi sağlanabilir.